DIGITAL SAFE LOCK

A Secure and Interactive Lock System with Visual Display on FPGA

Damini Katakam(23BEC0067)
Jyotsna Pillai Raghav (23BEC0076)
DV Miruthula (23BEC0064)
Harshitha N (23BML0091)

Introduction to Digital Lock Systems

- Definition: Digital lock systems are electronic systems designed to control access to secure areas or resources by verifying codes or patterns.
- Common uses: Homes, offices, safes, and restricted areas.
- Objective of the project: Create a simple digital lock system on an FPGA kit that displays "U" for unlock and "L" for lock based on switch inputs.

Project Objectives

- Main goals:
  - Design a digital lock system using verilog.
  - Implement the system on an FPGA (DE10-FPGA kit).
  - Use switch inputs to simulate the password or combination.
  - Display ‘U’ for unlocked and ‘L’ for locked states.
- User Interface: Controlled by 9 switches on the FPGA.

Design Overview

- FPGA components used:
  - Switches: To input the lock code.
  - 7-segment Display: To visually indicate ‘L’ for lock and ‘U’ for unlock.
  - FSM (Finite State Machine): Controls the state transitions based on inputs patterns.
- Behavioural and structural modeling:
  - Behavioural modelling for the lock/unlock control logic.
  - Structural modeling for handling the inputs and displaying the output states.

System Requirements

- Switch Configuration: 9 switches available for code input.
- Display logic: use 7-segment display to show ‘L’ for lock and ‘U’ for unlock.
- Security: Predefined password or sequence to unlock.
- Reset option: implement reset functionality to revert to the locked state.

Verilog Implementation

- Verilog modules:
  - Input decoder: reads switch inputs
  - Display controller: controls the display output (‘U’ or ‘L’)
- Key features: efficient coding for FPGA resources, minimal delay, easy debugging.

Simulation and testing

- Testbench setup: simulate the code entry and reset behaviour.
- Test scenarios:
  - Correct code input sequence.
  - Incorrect code attempts.
- Expected outcomes: showing simulation results confirming correct lock/unlock functionality.

```v
module digital_safe (
    input wire [3:0] entered_code,    // 4-bit input for the entered code
    input wire enter,                 // Signal to submit the entered code
    input wire reset,                 // Signal to reset the system
    output reg lock_state,            // Lock state (1 = locked, 0 = unlocked)
    output reg [6:0] seg              // 7-segment display output
);

 // Predefined passcode
 parameter PASSCODE = 4'b1010; // Set passcode to binary "1010" (10 in decimal)

 // Initial state is locked
initial begin
    lock_state = 1;
    seg = 7'b1111111; // Turn off all segments initially
end

 // Main control logic
always @(posedge enter or posedge reset) begin
    if (reset) begin
        lock_state <= 1;  // Resetting to locked state
    end else if (entered_code == PASSCODE) begin
        lock_state <= 0;  // Unlocking if passcode is correct
    end else begin
        lock_state <= 1;  // Staying locked if incorrect code
    end
end

 // 7-segment display driver for lock state
always @(lock_state) begin
```

```v
 parameter PASSCODE = 4'b1010; // Set passcode to binary "1010" (10 in decimal)

 // Initial state is locked
initial begin
    lock_state = 1;
    seg = 7'b1111111; // Turn off all segments initially
end

 // Main control logic
always @(posedge enter or posedge reset) begin
    if (reset) begin
        lock_state <= 1;  // Resetting to locked state
    end else if (entered_code == PASSCODE) begin
        lock_state <= 0;  // Unlocking if passcode is correct
    end else begin
        lock_state <= 1;  // Staying locked if incorrect code
    end
end

 // 7-segment display driver for lock state
always @(lock_state) begin
    case (lock_state)
        1: seg = 7'b1000111; // "L" for locked: segments b, c, d, e
        0: seg = 7'b1000001; // "U" for unlocked: segments a, b, c, d, e, f
        default: seg = 7'b1111111; // All segments off (shouldn't happen)
    endcase
end

endmodule
```

VERILOG CODE

```
module digital_safe (
    input wire [3:0] entered_code,    // 4-bit input for the entered code
    input wire enter,                 // Signal to submit the entered code
    input wire reset,                 // Signal to reset the system
    output reg lock_state,            // Lock state (1 = locked, 0 = unlocked)
    output reg [6:0] seg              // 7-segment display output
);

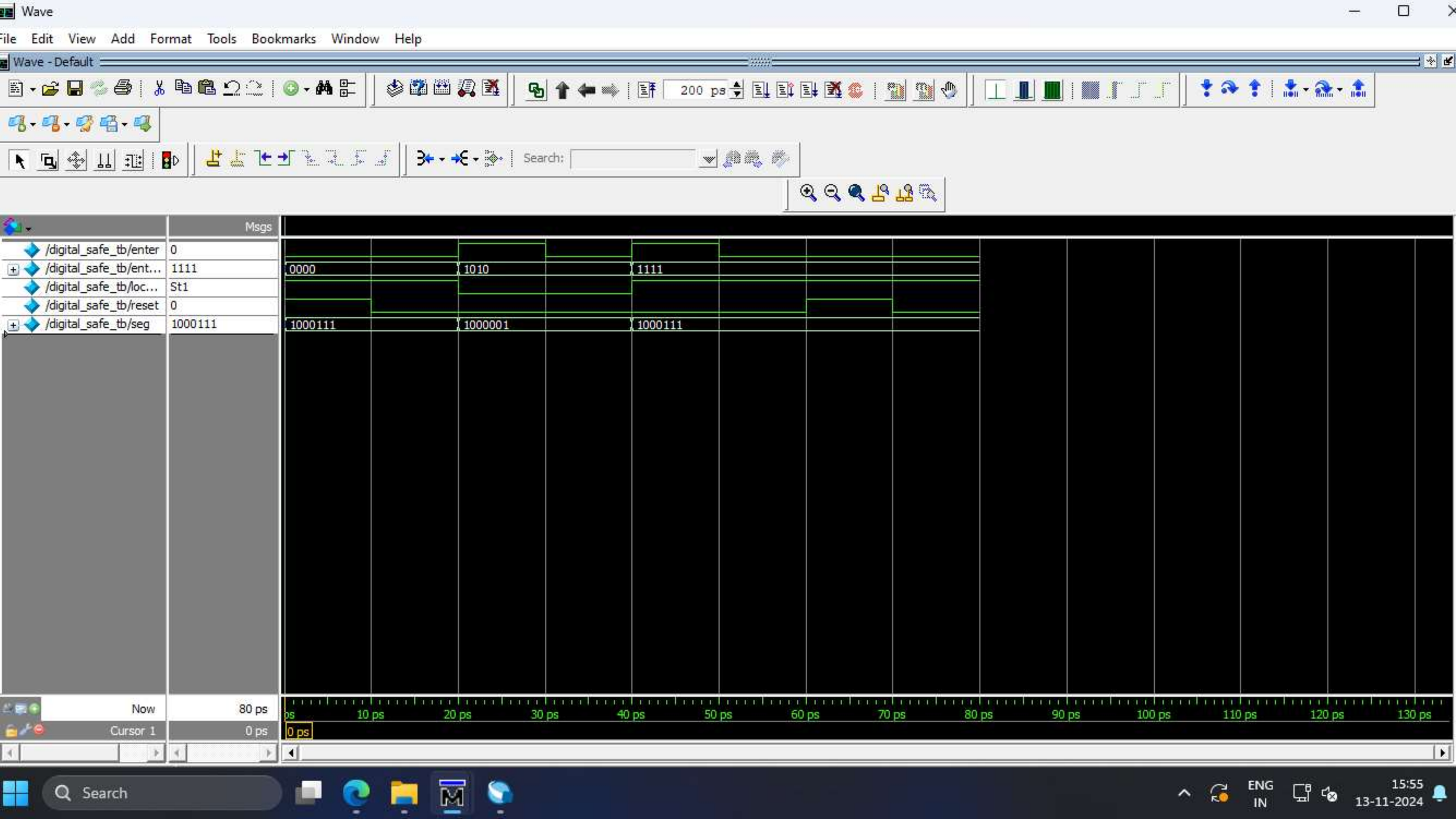
// Predefined passcode
parameter PASSCODE = 4'b1010; // Set passcode to binary "1010" (10 in decimal)

// Initial state is locked
initial begin
    lock_state = 1;
    seg = 7'b1111111; // Turn off all segments initially
end

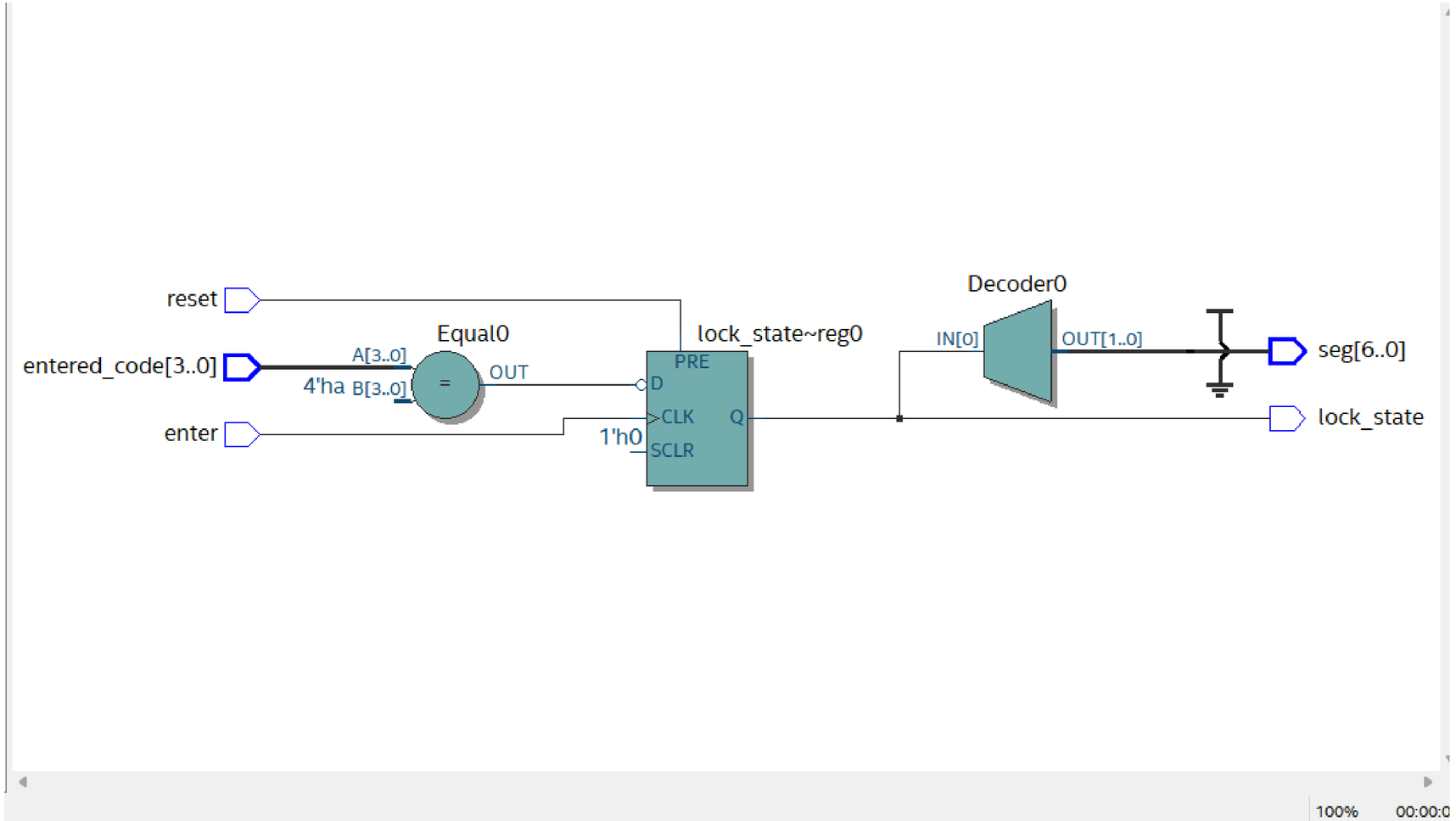
// Main control logic
always @(posedge enter or posedge reset) begin
    if (reset) begin
        lock_state <= 1;  // Resetting to locked state
    end else if (entered_code == PASSCODE) begin
        lock_state <= 0;  // Unlocking if passcode is correct
    end else begin
        lock_state <= 1;  // Staying locked if incorrect code
    end
end

// 7-segment display driver for lock state
always @(lock_state) begin
    case (lock_state)
        1: seg = 7'b1000111; // "L" for locked: segments b, c, d, e
        0: seg = 7'b1000001; // "U" for unlocked: segments a, b, c, d, e, f
        default: seg = 7'b1111111; // All segments off (shouldn’t happen)
    endcase
end

endmodule
```


Wave
File Edit View Add Format Tools Bookmarks Window Help
Wave - Default
200 ps
Search:
Msgs
/digital_safe_tb/enter
/digital_safe_tb/ent...
/digital_safe_tb/loc...
/digital_safe_tb/reset
/digital_safe_tb/seg
0
1111
St1
0
1000111
0000
1010
1111
1000111
1000001
1000111
Now
80 ps
Cursor 1
0 ps
0 ps
10 ps
20 ps
30 ps
40 ps
50 ps
60 ps
70 ps
80 ps
90 ps
100 ps
110 ps
120 ps
130 ps
Search
ENG
IN
15:55
13-11-2024

RTL VIEW


reset
entered_code[3..0]
enter
Equal0
A[3..0]
4'ha B[3..0]
=
OUT
lock_state~reg0
PRE
D
CLK
Q
1'h0
SCLR
Decoder0
IN[0]
OUT[1..0]
seg[6..0]
lock_state
100%
00:00:0

Implementation on FPGA

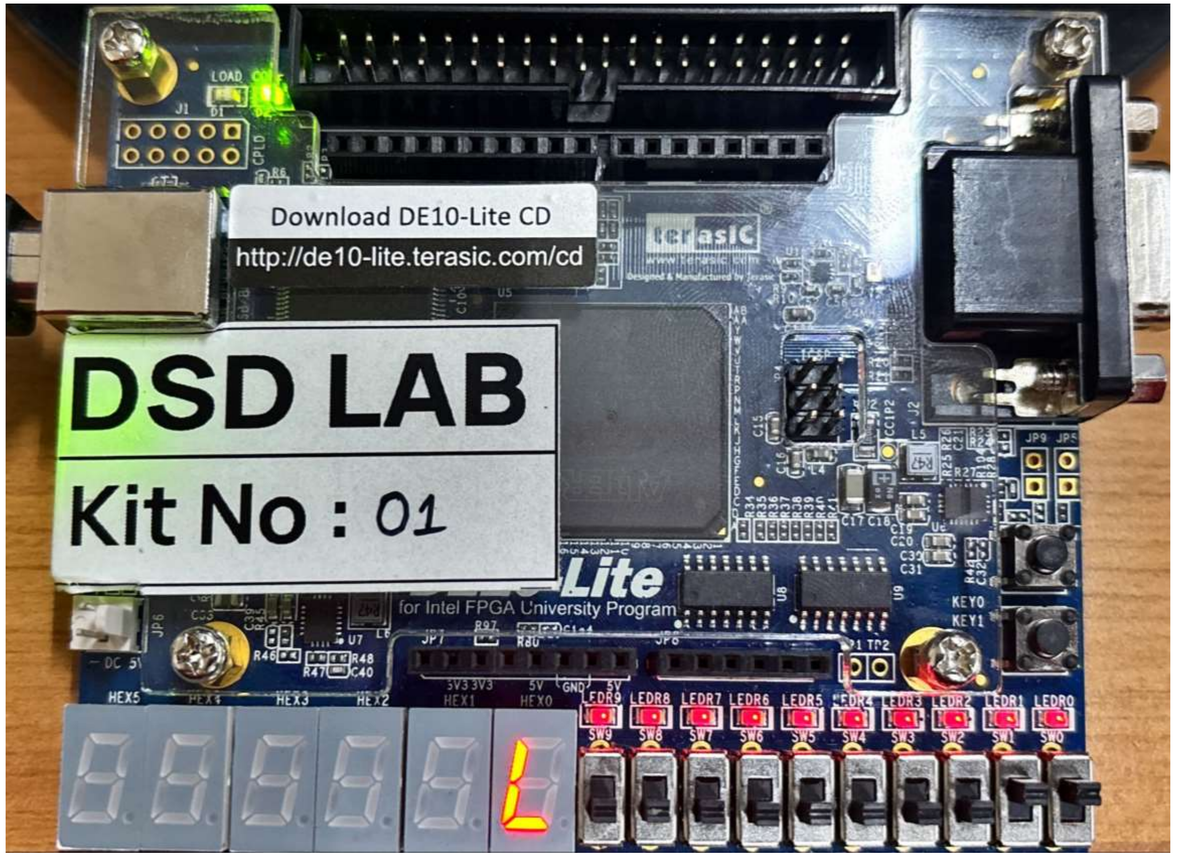
- DE10-FPGA Kit setup:

KEY[1]	enter
SW[0]	entered_code[0]
SW[1]	entered_code[1]
SW[2]	entered_code[2]
SW[3]	entered_code[3]
SW[4]	reset

HEX[0]	seg[0]
HEX[1]	seg[1]
HEX[2]	seg[2]
HEX[3]	seg[3]
HEX[4]	seg[4]
HEX[5]	seg[5]
HEX[6]	seg[6]



Input - 1010



Input - 0011

Challenges and Solutions

- Challenges:
  - Ensuring reliable switch debounce.
  - Display control for clear feedback
- Solutions:
  - Debounce logic.
  - Carefully tuned display drivers.

Conclusion

- Summary: the code is a digital lock status displayed on a 7-segment display and indicated by the lock_state variable.
  - Code output summary:
    - Lock-state is initially set to 1 (locked).
    - When the enter signal is triggered:
      - If the entered_code matched the predefined PASSCODE (4’b1010), lock_state is set to 0 (unlocked).
      - If the entered_code does not match, lock_state remains 1(locked).
    - When the reset signal is triggered, lock_state resets to 1 (locked), regardless the entered _code.
  - 7-segment display output (seg):
    - The display shows a specific character based on the lock_state.
      - ‘L’(locked): when lock_state is 1, segment b, c, d, e are lit up to display ‘L’ (binary 1000111).
      - ‘U’’(unlocked): when the lock_state is 0, segments a, b, c, d, e, and f are lip up to display ‘U’ (binary 1000001).
- This setup enables a visual representation of the lock status, showing ‘L’ when locked and ‘U’ when unlocked.

Q&A